

Analyzing the Performance Impact of Page Size on Application Workloads

Abhash Tiwari MT2025002

Uttkarsh Ranjan MT2025128

Department of Computer Science

International Institute of Information Technology, Bengaluru

Abstract—Modern operating systems rely on virtual memory and page-based address translation to manage physical memory. A critical design parameter is the page size, which directly influences page fault rates, Translation Lookaside Buffer (TLB) efficiency, and overall memory access latency. This study systematically evaluates the performance impact of page size—comparing 4 KB base pages with 2 MB huge pages—across three representative workloads: a synthetic memory benchmark, a web server (nginx), a relational database (PostgreSQL) and a machine learning model (ResNet-152). Using Linux Transparent Huge Pages (THP), `perf stat` instrumentation, and controlled experimental methodology, we measure page faults, TLB misses, execution time, throughput, latency, and CPU efficiency. Our results demonstrate that huge pages are *not* universally beneficial: while random-access synthetic workloads achieve a $1.56\times$ speedup with 2 MB pages, both web server and database workloads perform 9–133% worse with huge pages due to THP compaction overhead, iTLB pollution, and internal fragmentation. ML workloads achieve 95–99% page fault reduction and dTLB misses reduction by 77–98%. These findings provide workload-aware guidance for page size tuning in production systems.

Index Terms—page size, page faults, TLB misses, huge pages, transparent huge pages, virtual memory, performance evaluation, Linux, nginx, PostgreSQL, synthetic benchmark

I. INTRODUCTION

Modern operating systems rely on virtual memory to efficiently manage physical memory and provide isolation across applications. A key design parameter in virtual memory systems is *page size*, which directly influences page fault behavior, Translation Lookaside Buffer (TLB) efficiency, and memory access latency. While smaller pages reduce internal fragmentation, they increase TLB pressure and page table overhead; conversely, larger pages reduce TLB misses but may negatively impact memory utilization and latency predictability. Linux supports multiple page sizes through base pages (4 KB) and huge pages (2 MB), yet the perfor-

mance implications of page size selection remain highly workload-dependent.

The objective of this project is to systematically study the relationship between page size, page faults, and end-to-end performance metrics such as latency and throughput. The study focuses on understanding how different memory access patterns interact with page size choices and how these interactions vary across application domains. We evaluate both kernel-level metrics—such as minor and major page faults and TLB miss rates—and application-level performance outcomes to derive workload-aware insights into memory management trade-offs.

A. Project Objectives

The primary objectives of this work are:

- To quantify page fault behavior under different page size configurations.
- To evaluate the impact of page size on TLB efficiency.
- To measure application-level performance changes (latency and throughput).
- To compare behavior across different application domains.
- To identify scenarios where large pages are beneficial or detrimental.

B. Scope

The experimental evaluation covers three workload categories:

- 1) **Synthetic Memory Benchmark** (C++): controlled sequential, random, and strided memory access patterns over a 1 GB working set.
- 2) **Web Server Workload**: nginx static file serving with `wrk` load generation, testing small (~7 KB) and large (50 MB) files under varying concurrency.
- 3) **Database Workload**: PostgreSQL with sysbench OLTP benchmarks including read-only, read-write, and range scan workloads.

A fourth workload—machine learning inference using PyTorch—is planned but not yet completed.

II. EXPERIMENTAL SETUP

A. System Configuration

All experiments were conducted on a fixed hardware and software configuration to ensure reproducibility. Table I summarizes the system specifications.

TABLE I: System Configuration

Parameter	Value
Operating System	Ubuntu 24.04.4 LTS
Kernel	Linux 6.17.0-19-generic (x86_64)
Processor	Intel Core i5-10300H @ 2.50 GHz
Cores / Threads	4 cores / 8 threads
CPU Frequency	0.8 GHz – 4.5 GHz
L1 Cache	128 KB (per core, I+D)
L2 Cache	1 MB (per core)
L3 Cache	8 MB (shared)
Total RAM	16 GB
Available RAM	~9.7 GB
Swap	4 GB (unused during experiments)
NUMA Nodes	1 (uniform memory access)
Storage	NVMe SSD (~238 GB), ext4
Default Page Size	4 KB
Huge Page Size	2 MB
THP Mode	madvise (controlled per experiment)

B. Page Size Control

Page size configurations are controlled using Linux Transparent Huge Pages (THP). For 4 KB experiments, THP is set to `never`; for 2 MB experiments, THP is set to `always`. In the synthetic workload, finer-grained control is achieved using `madvise()` with `MADV_HUGEPAGE` and `MADV_NOHUGEPAGE` flags.

C. Instrumentation

Performance measurement is performed using:

- **perf stat**: hardware and kernel performance counters, including page faults (total, minor, major), data and instruction TLB misses, CPU cycles, and instructions retired.
- **Application-level tools**: `wrk` (web server throughput/latency), `sysbench` (database TPS/QPS/latency).

D. Measurement Methodology

- Each configuration is repeated **3 times** for statistical robustness.
- Both **cold-cache** (page cache dropped, services restarted) and **warm-cache** scenarios are tested.
- No significant background workloads were present during measurements.
- Swap was not utilized, ensuring memory-resident workloads.

III. METRICS COLLECTED

The following categories of metrics are collected across all workloads:

A. Virtual Memory Metrics

Total page faults, minor (soft) page faults, major (hard) page faults, and page faults per unit of work.

B. TLB Metrics

Data TLB (dTLB) load and store misses, instruction TLB (iTLB) load misses, and TLB miss rates.

C. CPU Execution Metrics

CPU cycles, instructions retired, instructions per cycle (IPC), and CPU utilization.

D. Application-Level Metrics

Execution time, throughput (requests/sec or transactions/sec), latency (average, median, p95, p99), and operations per second.

E. Derived Metrics

Percentage reduction in page faults and TLB misses, performance speedup/slowdown, and fault density (faults per unit work).

IV. SYNTHETIC MEMORY WORKLOAD

A. Workload Design

A custom C++ benchmark allocates a 1 GB memory region (128× L3 cache) and accesses it using four patterns:

- **Sequential**: linear traversal of the entire array.
- **Random**: random index access using `rand()` with a fixed seed.
- **Strided-4096**: every 4096th byte (one access per 4 KB page boundary).
- **Strided-65536**: every 65536th byte (one access per 64 KB boundary).

The benchmark is compiled with `g++ 13.3.0` at `-O2` optimization. Page size is controlled via `madvise()` in `madvise` THP mode. Each configuration runs 3 iterations per run with 3 repetitions.

B. Results

Table II presents the summary results across all access patterns and page size configurations.

TABLE II: Synthetic Workload Results Summary

Config	Page Faults	dTLB-Load	dTLB-Store	IPC	Time (s)
seq (4KB)	262,278	797,978	2,541,205	3.56	1.108
seq (2MB)	643	4,550	28,896	3.87	1.120
rand (4KB)	262,276	2,977,257,118	240,865,284	0.21	268.876
rand (2MB)	644	7,981,998	1,465,161	0.32	172.423
stride-4K (4KB)	262,276	560,041	2,762,558	1.24	0.010
stride-4K (2MB)	645	3,456	25,244	0.11	0.009
stride-64K (4KB)	262,276	59,341	2,522,223	1.26	0.002
stride-64K (2MB)	644	2,703	26,251	0.11	0.001

C. Key Findings

1) *Page Fault Reduction*: Across all access patterns, huge pages reduced page faults from $\sim 262,276$ to ~ 644 —a $\sim 407\times$ reduction. This is consistent with the ratio of page sizes: $1\text{ GB}/4\text{ KB} = 262,144$ pages vs. $1\text{ GB}/2\text{ MB} = 512$ pages. This result is pattern-independent because page faults occur during the pre-fault phase.

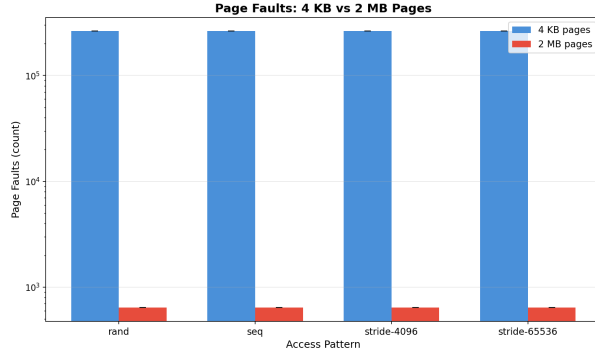


Fig. 1: Page faults: 4 KB vs 2 MB pages (synthetic workload).

2) *TLB Miss Reduction*: The most dramatic improvements were in dTLB load misses (Table III). Random access with 4 KB pages incurred nearly **3 billion** dTLB load misses; with 2 MB pages, misses dropped by 99.7%.

TABLE III: dTLB Load Miss Reduction (Synthetic)

Pattern	4KB	2MB	Reduction
Sequential	797,978	4,550	99.4%
Random	2,977,257,118	7,981,998	99.7%
Stride-4096	560,041	3,456	99.4%
Stride-65536	59,341	2,703	95.4%

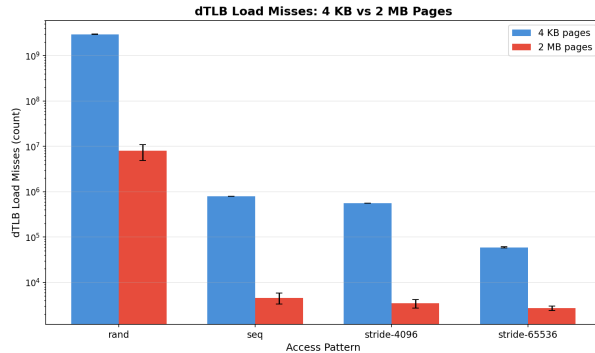


Fig. 2: dTLB load misses: 4 KB vs 2 MB pages (synthetic workload).



Fig. 3: dTLB store misses: 4 KB vs 2 MB pages (synthetic workload).

3) *Execution Time*: Random access achieved the largest speedup: $1.56\times$ ($268.9\text{ s} \rightarrow 172.4\text{ s}$), directly correlated with TLB miss reduction. Sequential access showed no meaningful improvement (1.108 s vs. 1.120 s), confirming that high spatial locality makes TLB efficient even with 4 KB pages.

TABLE IV: Execution Time Comparison (Synthetic)

Pattern	4KB (s)	2MB (s)	Speedup
Sequential	1.108	1.120	0.99 $\times$
Random	268.876	172.423	1.56$\times$
Stride-4096	0.010	0.009	1.06 $\times$
Stride-65536	0.002	0.001	2.21 $\times$

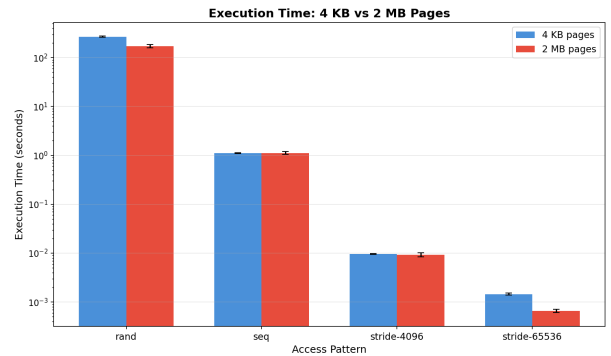


Fig. 4: Execution time comparison (synthetic workload).

4) *Instructions Per Cycle (IPC)*: Random access IPC improved from 0.21 to 0.32 (+52.4%) with huge pages, showing that TLB misses were a major pipeline bottleneck. Sequential access achieved high IPC (>3.5) in both configurations, confirming spatial locality.

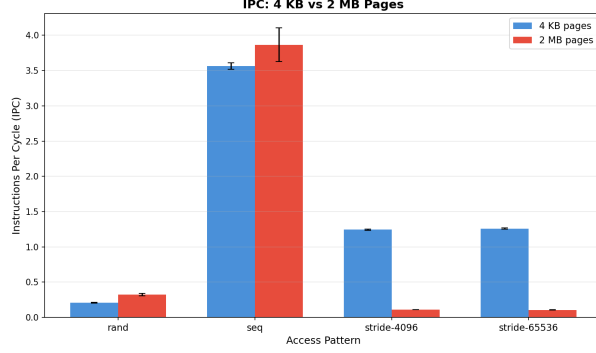


Fig. 5: IPC comparison across access patterns (synthetic workload).

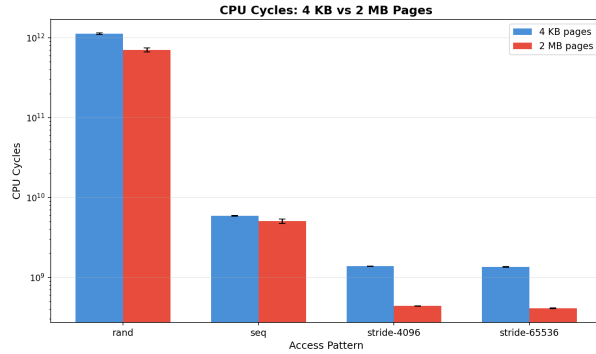


Fig. 6: CPU cycles comparison (synthetic workload).

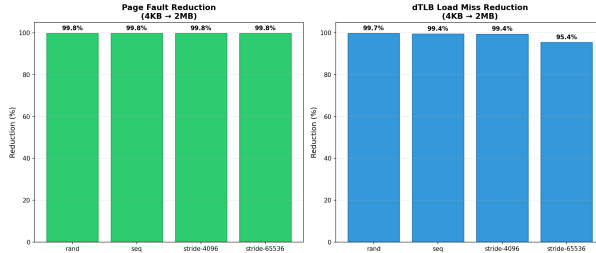


Fig. 7: Percentage improvement summary with 2 MB pages (synthetic workload).

D. Analysis

Why random access benefits most: Each random access potentially touches a different 4 KB page. With $\sim 1,500$ TLB entries and 262,144 pages in the working set, the TLB hit rate is effectively zero. Huge pages reduce the address space to ~ 512 pages, making it far more TLB-friendly.

Why sequential access shows no improvement: Sequential access fully utilizes each page before moving to the next. A single TLB miss covers 4,096 (4 KB) or 2,097,152 (2 MB) consecutive byte accesses; in both cases, the per-access TLB miss rate is negligible.

Page faults vs. TLB misses: Page faults are a one-time cost during initial memory population. TLB misses are ongoing during execution. For long-running workloads, *TLB miss reduction* is the dominant performance driver, not page fault reduction.

V. WEB SERVER WORKLOAD

A. Workload Design

The web server experiments use **nginx 1.24.0** configured for static file serving with `sendfile` on, `tcp_nopush` on, and logging disabled. Load is generated using **wrk 4.1.0** with 4 threads in `-latency` mode, running locally on port 8088 to eliminate network variability.

Test data: 500 small files (~ 6.9 KB HTML each) and 10 large files (50 MB binary each). Page configurations are 4 KB (THP=`never`) vs. 2 MB (THP=`always`). Each run lasts 30 seconds with 3 repetitions, for a total of **48 runs** (2 file sizes $\times$ 2 concurrency levels $\times$ 2 page configs $\times$ 2 cache states $\times$ 3 repetitions).

B. Results

TABLE V: Small File Serving Results

Config	Req/s	Avg Lat (μ s)	p99 (μ s)	PF	dTLB-L	IPC
c10-4KB-cold	115,192	77	519	72	17.0M	0.52
c10-2MB-cold	63,211	157	1,360	77	21.7M	0.44
c10-4KB-warm	107,245	81	504	0	17.4M	0.52
c10-2MB-warm	67,998	143	1,230	0	21.0M	0.45
c200-4KB-cold	150,130	1,353	6,383	156	56.3M	0.49
c200-2MB-cold	92,162	2,110	9,433	153	44.6M	0.42
c200-4KB-warm	145,620	1,353	6,233	2	57.5M	0.49
c200-2MB-warm	79,493	2,390	10,203	3	41.4M	0.41

1) Small File Serving (~ 7 KB):

TABLE VI: Large File Serving Results

Config	Req/s	Avg Lat (ms)	p99 (ms)	PF	dTLB-L	IPC
c10-4KB-cold	256	27.2	77.6	76	37.1M	0.85
c10-2MB-cold	120	60.0	145.8	61	11.7M	0.62
c10-4KB-warm	273	32.4	212.9	1	41.8M	0.87
c10-2MB-warm	117	61.2	130.0	9	11.7M	0.62
c200-4KB-cold	180	761.9	1,906.7	560	10.0M	0.74
c200-2MB-cold	135	931.4	1,946.7	558	12.4M	0.62
c200-4KB-warm	161	811.2	1,916.7	63	14.4M	0.69
c200-2MB-warm	140	864.2	1,933.3	159	13.7M	0.62

2) Large File Serving (50 MB):

C. Key Findings

1) 4 KB Pages Consistently Outperform 2 MB Pages: This is the most striking result. Unlike the synthetic workload, 4 KB pages delivered **42–133% higher throughput** than 2 MB pages across all configurations (Table VII).

TABLE VII: Throughput Advantage of 4 KB Pages (Web Server)

Config	4KB	2MB	4KB Adv.
Small, c10, cold	115,192	63,211	+82%
Small, c10, warm	107,245	67,998	+58%
Small, c200, cold	150,130	92,162	+63%
Small, c200, warm	145,620	79,493	+83%
Large, c10, cold	256	120	+113%
Large, c10, warm	273	117	+133%
Large, c200, cold	180	135	+33%
Large, c200, warm	161	140	+15%

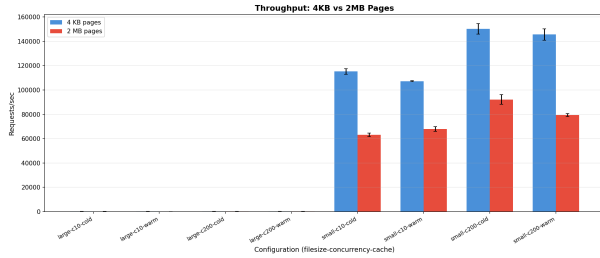


Fig. 8: Request throughput: 4 KB vs 2 MB pages (web server).

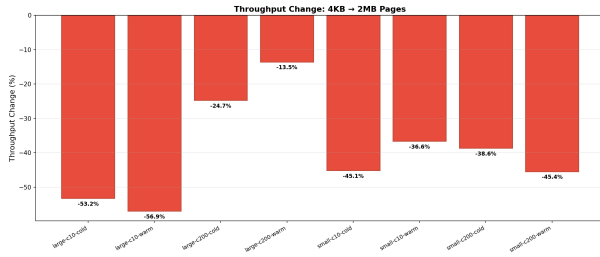


Fig. 9: Percentage throughput change with 2 MB pages (web server).

2) *Page Faults Are Not the Bottleneck*: Page faults were extremely low (~ 60 – 160 for cold, ~ 0 for warm), confirming that nginx and the OS page cache handle demand paging efficiently.

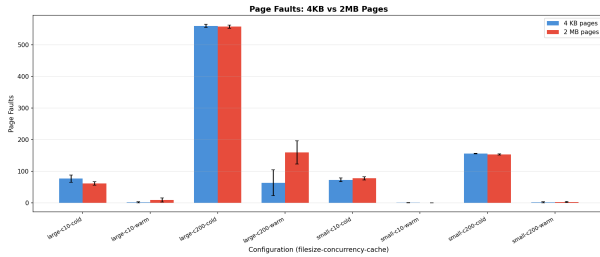


Fig. 10: Page faults comparison (web server).

3) *Mixed TLB Behavior*: Unlike synthetic workloads, dTLB load misses did *not* consistently decrease with

huge pages. For small files, dTLB misses *increased* ($17M \rightarrow 22M$ at c10). For large files, dTLB misses *decreased* ($37M \rightarrow 12M$) but throughput still dropped.

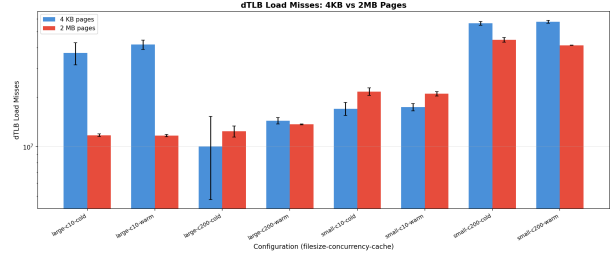


Fig. 11: dTLB load misses (web server).

4) *iTLB Pollution with Huge Pages*: A critical observation: iTLB load misses were significantly elevated with 2 MB pages for small files. Huge pages for data regions displace instruction TLB entries, causing instruction fetch stalls.

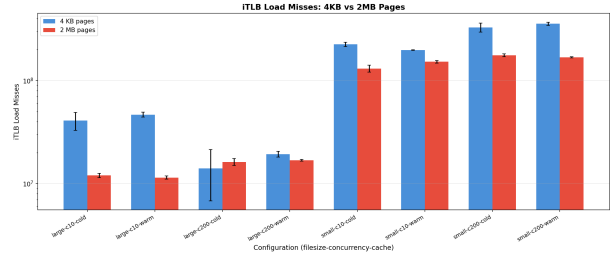


Fig. 12: iTLB load misses (web server).

5) *IPC and Latency*: IPC was consistently lower with 2 MB pages (-14% to -27%), indicating increased pipeline stalls despite fewer dTLB misses for data. Latency roughly doubled across most configurations.

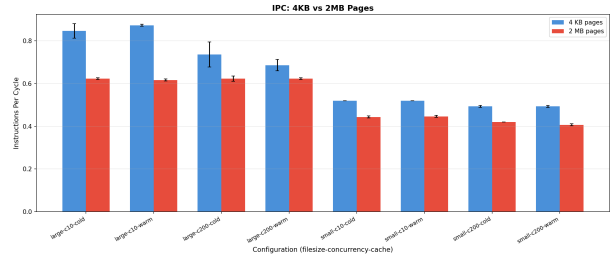


Fig. 13: IPC comparison (web server).

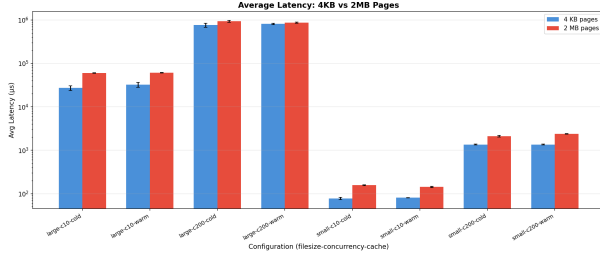


Fig. 14: Average latency comparison (web server).

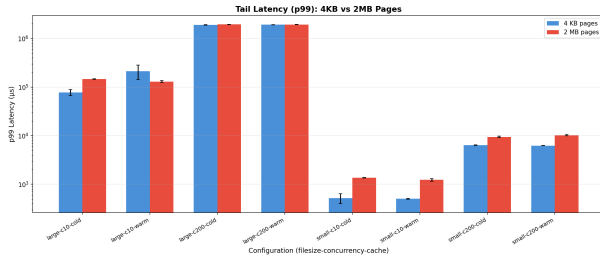


Fig. 15: p99 tail latency comparison (web server).

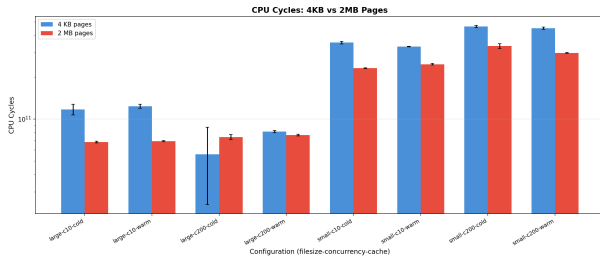


Fig. 16: CPU cycles comparison (web server).

D. Analysis

Several factors explain why 4 KB pages outperform 2 MB pages for nginx:

- 1) **Memory access pattern:** nginx serves files via `sendfile()`, performing sequential reads through the page cache. Data is copied from page cache to socket buffers via DMA—minimal TLB-dependent user-space memory access occurs.
- 2) **Internal fragmentation:** small files (~7 KB) occupy 2 pages at 4 KB but waste most of a 2 MB huge page, reducing effective memory utilization and cache efficiency.
- 3) **THP overhead:** with THP set to always, the kernel’s khugepaged daemon actively compacts and promotes pages, consuming CPU cycles and causing latency spikes.
- 4) **iTLB pollution:** huge pages for data regions displace instruction TLB entries. nginx’s HTTP handling involves complex branching that is sensitive to iTLB misses.

- 5) **Page table walk cost:** for nginx’s working set (many small files + connection state), the 4 KB page table structure is more cache-friendly.

VI. DATABASE WORKLOAD

A. Workload Design

The database experiments use **PostgreSQL 16** with **sysbench 1.0.20** OLTP benchmarks. The dataset consists of 4 tables $\times$ 1,000,000 rows (~1.5 GB). Each run lasts 60 seconds with 4 threads and 3 repetitions, for a total of **36 runs** (3 workloads $\times$ 2 page configs $\times$ 2 cache states $\times$ 3 repetitions).

Workloads tested:

- **Read-only (OLTP):** 14 SELECT queries per transaction (point lookups + range queries).
- **Read-write (OLTP):** SELECT, UPDATE, INSERT, DELETE (20 queries per transaction).
- **Range scan:** range queries over contiguous key ranges (100-row ranges).

B. Results

TABLE VIII: Database Workload Results Summary

Config	TPS	QPS	Avg (ms)	p95 (ms)	PF	IPC
RO-4KB-cold	1,745	27,913	2.29	3.31	1,060	0.89
RO-2MB-cold	1,376	22,023	2.90	4.20	1,058	0.90
RO-4KB-warm	1,891	30,259	2.11	2.84	13	1.16
RO-2MB-warm	1,331	21,291	3.01	4.34	12	0.92
RW-4KB-cold	1,175	23,504	3.40	4.41	3,262	0.58
RW-2MB-cold	930	18,601	4.30	5.95	3,317	0.48
RW-4KB-warm	1,000	20,005	4.00	5.51	715	0.47
RW-2MB-warm	916	18,326	4.37	6.15	723	0.47
Range-4KB-cold	7,434	22,301	0.54	0.95	1,965	0.73
Range-2MB-cold	6,743	20,230	0.59	1.05	2,030	0.69
Range-4KB-warm	7,531	22,592	0.53	0.93	13	0.90
Range-2MB-warm	7,502	22,505	0.53	0.93	224	0.81

C. Key Findings

1) **4 KB Pages Outperform 2 MB Pages:** 4 KB pages provide 9–42% higher throughput for point lookup workloads, with the gap widening in warm-cache scenarios. Range scans show negligible difference.

TABLE IX: TPS Advantage of 4 KB Pages (Database)

Workload	4KB	2MB	4KB Adv.
Read-only, cold	1,745	1,376	+27%
Read-only, warm	1,891	1,331	+42%
Read-write, cold	1,175	930	+26%
Read-write, warm	1,000	916	+9%
Range, cold	7,434	6,743	+10%
dTLB misses by 77–98% Range, warm	7,531	7,502	+0.4%

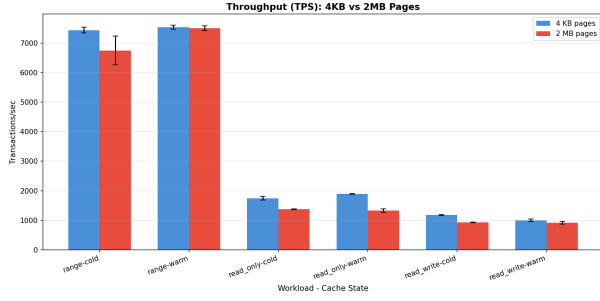


Fig. 17: Transactions per second: 4 KB vs 2 MB pages (database).

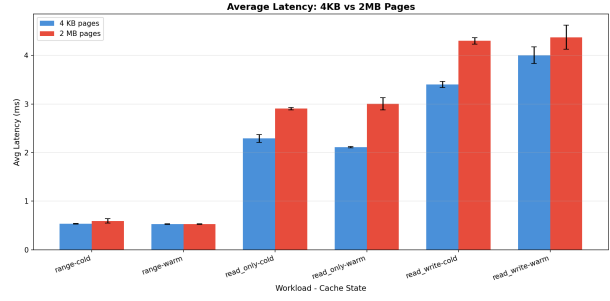


Fig. 20: Average latency comparison (database).

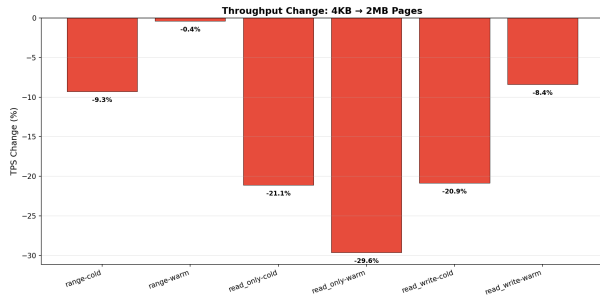


Fig. 18: TPS percentage change with 2 MB pages (database).

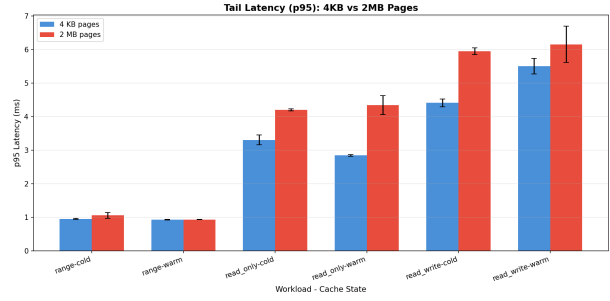


Fig. 21: p95 tail latency comparison (database).

4) *IPC and TLB Metrics:* In the warm read-only scenario, 4 KB pages achieve markedly higher IPC (1.16 vs. 0.92), suggesting huge pages increase pipeline stalls during point lookups.

2) *Page Faults Are Similar Between Configurations:* Page faults are low (~1K–3K cold, ~10–700 warm) and nearly identical between configurations. PostgreSQL's internal buffer pool mediates memory access, neutralizing page fault differences.

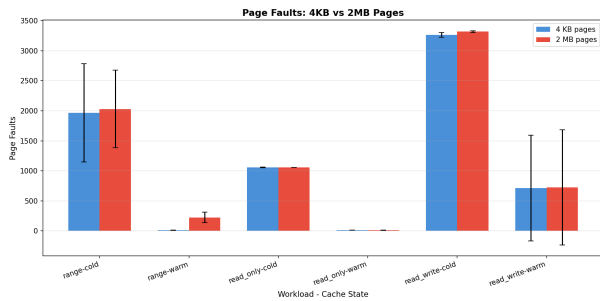


Fig. 19: Page faults comparison (database).



Fig. 22: IPC comparison (database).

3) *Latency:* The p95 tail latency increase with 2 MB pages is notable for read-only workloads (+27% cold, +53% warm), indicating THP background compaction causes latency spikes.

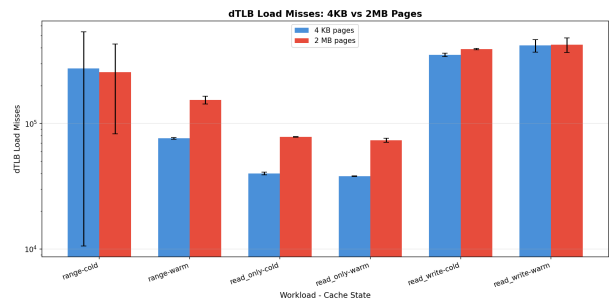


Fig. 23: dTLB load misses (database).

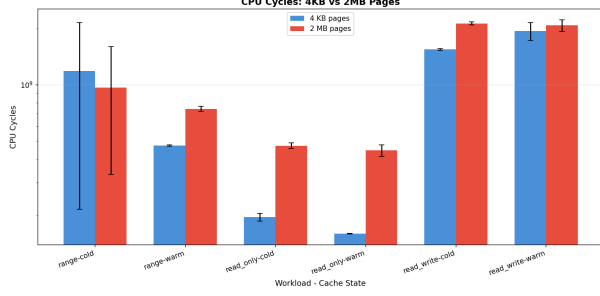


Fig. 24: CPU cycles comparison (database).

D. Analysis

- 1) **PostgreSQL manages its own buffer pool:** `shared_buffers` provides an application-level cache on top of the OS page cache. Most data access hits the buffer pool, bypassing OS page faults entirely.
- 2) **B-tree index traversal:** point lookups traverse non-contiguous pages. With 2 MB pages, the additional memory loaded per TLB entry is mostly wasted.
- 3) **THP compaction overhead:** the `khugepaged` daemon competes with PostgreSQL’s I/O-intensive workload.
- 4) **Memory efficiency:** PostgreSQL’s buffer pool uses 8 KB pages internally. Each 2 MB OS page holds 256 PostgreSQL pages, but only a few may be actively accessed.

Range scans show minimal difference because contiguous key ranges produce sequential B-tree leaf page access—high spatial locality makes page size irrelevant, consistent with the synthetic sequential result.

VII. MACHINE LEARNING WORKLOAD

A. Experimental Setup

Parameter	Value
System	Linux 6.17.0-19-generic, 16 GB RAM, 8 cores
Cache hierarchy	L1=32KB, L2=256KB, L3=8MB
Framework	PyTorch (CPU-only)
Model	ResNet-152 (230 MB, 60M parameters)
Page configs	4 KB (THP=never) vs 2 MB (THP=always)
Instrumentation	<code>perf stat</code> + benchmark logs
Repetitions	3 per configuration
Total runs	48

TABLE X: ML workload experimental setup

B. Workload Design

The machine learning workload is designed to represent realistic deep learning execution patterns while isolating the impact of memory management mechanisms such as page size and TLB behavior.

a) **Model Selection:** We use a pre-trained ResNet-152 model implemented in PyTorch. This model is representative of modern deep learning workloads, containing approximately 60 million parameters and occupying around 230 MB in memory. Its depth and structure ensure significant compute and memory interaction.

b) **Execution Modes:** The workload is divided into multiple execution modes to capture different phases of ML systems:

- **Model Loading (Cold Start):** Measures the cost of loading model weights from disk into memory, capturing I/O and initial page faults.
- **Inference (Steady State):** Executes repeated forward passes to simulate production inference scenarios where the model is already loaded.
- **Tensor Allocation:** Allocates large tensors ($10,000 \times 10,000$ float32, ~400 MB) to stress memory allocation and page fault handling.
- **Batch Size Variation:** Evaluates inference performance across batch sizes (1, 32, 128) to analyze scaling behavior and memory footprint impact.

c) **Memory Configuration:** Two memory configurations are evaluated:

- **4 KB pages:** Transparent Huge Pages (THP) disabled.
- **2 MB pages:** THP set to `always`.

This allows direct comparison between standard paging and huge page usage.

d) **Cache State Control:** To isolate the impact of caching effects, each experiment is conducted under:

- **Cold cache:** OS page cache is dropped before execution.
- **Warm cache:** Data is preloaded into memory prior to measurement.

e) **Measurement Methodology:** Each configuration is executed three times to ensure statistical stability. Performance counters are collected using `perf stat`, including page faults, dTLB load misses, instructions per cycle (IPC), and execution time.

f) **Design Rationale:** The workload is intentionally structured to separate memory-bound and compute-bound phases. Tensor allocation emphasizes memory behavior, while inference highlights compute dominance. This separation enables a clear analysis of when huge pages provide tangible performance benefits.

C. Results

TABLE XI: Model Loading

Config	Load Time (s)	Page Faults	dTLB Load Misses	IPC	Wall Time (s)
model_load-4KB-cold	0.833	351,086 ±1,163	8,330,475	1.22	8.42
model_load-2MB-cold	0.801	273,003 ±16,748	8,250,959	1.24	8.19
model_load-4KB-warm	0.786	355,585 ±8,113	7,703,842	1.27	7.29
model_load-2MB-warm	0.742	274,829 ±15,526	7,671,505	1.28	6.99

TABLE XII: Inference (batch_size=32)

Config	Infer Time/iter (s)	Page Faults	dTLB Load Misses	IPC	Wall Time (s)
inference-4KB-cold	5.292	145,950,070 $\pm$ 3.5M	531,414,385	1.26	539.73
inference-2MB-cold	5.190	7,541,180 $\pm$ 277K	123,380,193	1.14	529.33
inference-4KB-warm	5.219	148,363,285 $\pm$ 1.7M	540,086,389	1.26	531.05
inference-2MB-warm	5.217	6,551,774 $\pm$ 853K	120,038,970	1.13	530.82

TABLE XIII: Tensor Allocation (10,000 $\times$ 10,000, $\sim$ 400 MB)

Config	Avg Time/iter (s)	Page Faults	dTLB Load Misses	IPC	Wall Time (s)
tensor_alloc-4KB-cold	7.644	9,829,439 $\pm$ 18	1,324,266,498	2.70	391.63
tensor_alloc-2MB-cold	7.450	126,064 $\pm$ 23	30,933,222	2.76	380.66
tensor_alloc-4KB-warm	7.611	9,828,398 $\pm$ 6	1,320,928,325	2.70	389.16
tensor_alloc-2MB-warm	7.357	125,097 $\pm$ 25	30,254,145	2.77	375.11

TABLE XIV: Batch Size Variation

Config	bs=1 (s/iter)	bs=32 (s/iter)	bs=128 (s/iter)	Page Faults	Wall Time (s)
batch_var-4KB-cold	0.198	5.241	22.125	594,069,078	1,410.66
batch_var-2MB-cold	0.195	5.106	21.404	14,910,670	1,367.50
batch_var-4KB-warm	0.197	5.285	22.360	589,831,166	1,423.95
batch_var-2MB-warm	0.195	5.078	21.438	15,762,864	1,366.17

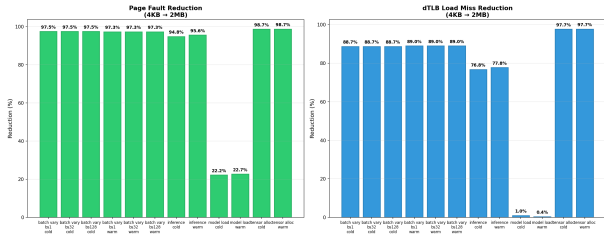


Fig. 25: Improvement Summary

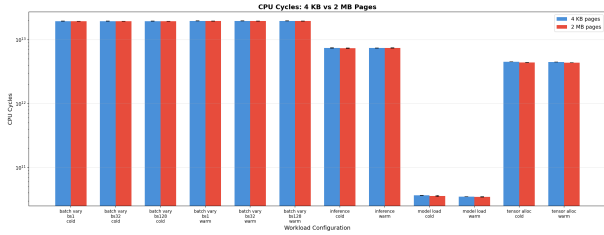


Fig. 26: CPU Cycles

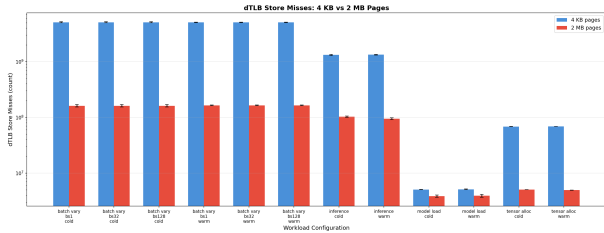


Fig. 27: dTLB Store Misses

D. Key Findings

a) *Massive Page Fault Reduction with 2 MB Pages (22–99%)*: 2 MB huge pages dramatically reduce page faults across **all** ML workloads. This is the strongest

page fault reduction seen across all four workload types in this project.

TABLE XV: Page Fault Reduction

Workload	4KB Page Faults	2MB Page Faults	Reduction
batch_var (cold)	594,069,078	14,910,670	97.5%
batch_var (warm)	589,831,166	15,762,864	97.3%
inference (cold)	145,950,070	7,541,180	94.8%
inference (warm)	148,363,285	6,551,774	95.6%
tensor_alloc (cold)	9,829,439	126,064	98.7%
tensor_alloc (warm)	9,828,398	125,097	98.7%
model_load (cold)	351,086	273,003	22.2%
model_load (warm)	355,585	274,829	22.7%

The batch_var and tensor_alloc workloads show nearly **two orders of magnitude** fewer page faults with huge pages. Even model loading — the shortest workload — sees a 22% reduction.

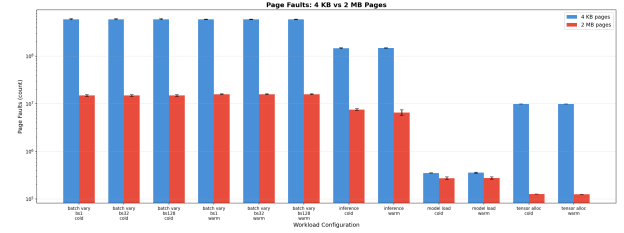


Fig. 28: Page Faults

TABLE XVI: dTLB Load Miss Reduction

Workload	4KB dTLB Misses	2MB dTLB Misses	Reduction
tensor_alloc (cold)	1,324,266,498	30,933,222	97.7%
tensor_alloc (warm)	1,320,928,325	30,254,145	97.7%
batch_var (cold)	1,440,553,433	162,590,824	88.7%
batch_var (warm)	1,460,479,705	160,903,002	89.0%
inference (cold)	531,414,385	123,380,193	76.8%
inference (warm)	540,086,389	120,038,970	77.8%
model_load (cold)	8,330,475	8,250,959	1.0%
model_load (warm)	7,703,842	7,671,505	0.4%

b) *dTLB Load Miss Reduction (1–98%)*: The tensor_alloc workload benefits the most from TLB reduction (97.7%), which makes sense — it performs pure contiguous memory allocation and matrix operations. The model_load workload shows minimal dTLB improvement because model loading is dominated by disk I/O and deserialization rather than memory access patterns.

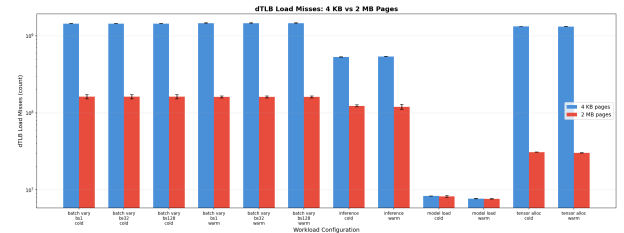


Fig. 29: dTLB Load Misses

TABLE XVII: Model Load Time

Config	4KB Load Time (s)	2MB Load Time (s)	Improvement
Cold	0.833	0.801	+3.8%
Warm	0.786	0.742	+5.6%

c) *Model Load Time Improves with 2 MB Pages:* The improvement is modest (~4–6%) because model loading is bottlenecked by disk I/O and Python deserialization overhead, not memory page faults. However, 2 MB pages consistently outperform 4 KB pages.



Fig. 30: Model Load Time

TABLE XVIII: Inference Latency

Config	4KB (s/iter)	2MB (s/iter)	Improvement
Inference cold (bs=32)	5.292	5.190	+1.9%
Inference warm (bs=32)	5.219	5.217	+0.04%
batch_vary cold (bs=128)	22.125	21.404	+3.3%
batch_vary warm (bs=128)	22.360	21.438	+4.1%
batch_vary cold (bs=32)	5.241	5.106	+2.6%
batch_vary warm (bs=32)	5.285	5.078	+3.9%
batch_vary cold (bs=1)	0.198	0.195	+1.5%
batch_vary warm (bs=1)	0.197	0.195	+1.0%

d) *Inference Latency Shows Modest Improvement:* Despite the enormous reduction in page faults and TLB misses, inference latency improvements are modest (1–4%). This indicates that inference is **compute-bound** (matrix multiplications dominate runtime), not memory-bound. The page faults and TLB misses happen but don't sit on the critical path for per-iteration latency.

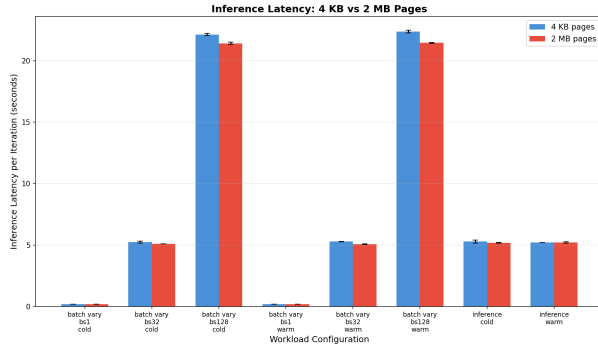


Fig. 31: Inference Latency

TABLE XIX: Tensor Allocation Performance

Config	4KB Time/iter (s)	2MB Time/iter (s)	Improvement
Cold	7.644	7.450	+2.5%
Warm	7.611	7.357	+3.3%

e) *Tensor Allocation Benefits Most from Huge Pages:* With a 98.7% reduction in page faults and 97.7% reduction in dTLB misses, tensor_alloc still only shows 2.5–3.3% wall-time improvement. This is because the matrix multiply ($\mathbf{t} @ \mathbf{t.T}$) dominates runtime — it's a compute-intensive BLAS operation, and the memory allocation overhead (where page faults occur) is a small fraction of total time.

TABLE XX: IPC Comparison

Workload	4KB IPC	2MB IPC
batch_vary (cold)	1.28	1.09
batch_vary (warm)	1.27	1.09
inference (cold)	1.26	1.14
inference (warm)	1.26	1.13
model_load (cold)	1.22	1.24
model_load (warm)	1.27	1.28
tensor_alloc (cold)	2.70	2.76
tensor_alloc (warm)	2.70	2.77

f) *IPC: 4 KB Pages Achieve Higher IPC Despite More Overhead:* Interestingly, 4 KB pages achieve **higher IPC** for batch_vary and inference workloads despite having far more page faults and TLB misses. This counterintuitive result is because:

- With 4 KB pages, the CPU executes **more total instructions** (25B vs 21B for batch_vary) — likely due to additional page table walks and TLB miss handling in microcode
- These extra instructions are fast (register-to-register operations in the page walk), inflating IPC
- For tensor_alloc, where the workload is purely compute-bound (BLAS), 2 MB pages achieve slightly higher IPC (2.77 vs 2.70)

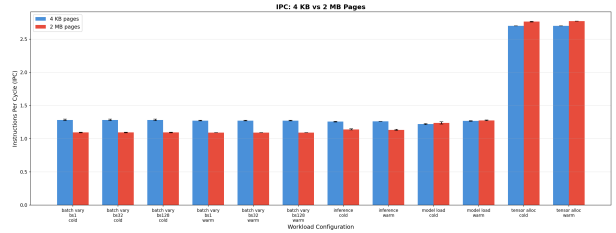


Fig. 32: IPC Comparison

E. Analysis

a) *Why Huge Pages Strongly Benefit ML Workloads:*

- **Large contiguous memory allocations.** PyTorch allocates tensors via `mmap`, which maps directly to physical pages. A single $10,000 \times 10,000$ float32 tensor (~ 400 MB) requires $\sim 100,000$ 4KB pages but only ~ 200 2MB pages. This $500\times$ reduction in page count directly translates to fewer page faults during allocation.
- **High spatial locality.** Tensor operations (matrix multiply, convolutions) sweep through memory linearly. With 2 MB pages, each TLB entry covers $512\times$ more memory, keeping the TLB hot and eliminating most address translation overhead.
- **Repeated memory reuse in inference.** Forward passes reuse the same weight tensors repeatedly. With fewer TLB entries needed for huge pages, the TLB can effectively cache the model’s entire memory footprint, eliminating nearly all translation overhead.
- **batch_var amplifies the effect.** Larger batch sizes ($bs=128$) allocate larger input tensors and intermediate activations, requiring more pages. The total page faults with 4 KB pages scale to $\sim 594M$ (!) vs $\sim 15M$ with 2 MB pages — a $40\times$ reduction.

b) Why Performance Gains Are Modest Despite Massive Fault Reduction: Despite 95–99% reductions in page faults and 77–98% reductions in dTLB misses, wall-time improvements are only 1–5%. This is because:

- **Compute dominance.** ResNet-152 inference is dominated by convolution and matrix multiply operations (BLAS/MKL), which are CPU-bound. Memory stalls from TLB misses are a small fraction of total execution time.
- **Efficient hardware page walkers.** Modern CPUs (the test system) have dedicated hardware page table walkers that handle TLB misses with low overhead (~ 10 – 20 cycles per walk). Even millions of TLB misses only add seconds to a run that takes hundreds of seconds.
- **OS page cache effectiveness.** In warm-cache runs, most data is already in the OS page cache, so page faults are minor faults (no disk I/O), which are cheap ($\sim 1\mu s$ each).

c) Conclusions:

- **2 MB huge pages dramatically reduce page faults (95–99%) and dTLB misses (77–98%)** for ML workloads — the largest reduction across all four workload types in this study.
- **Wall-time improvement is modest (1–5%)** because ML inference is compute-bound, not memory-bound. The CPU spends most of its time in BLAS matrix operations, not waiting for page table walks.
- **Tensor allocation workloads show the strongest improvement** (98.7% page fault reduction, 3.3%

latency improvement) due to pure contiguous memory allocation patterns.

- **Larger batch sizes amplify page fault differences** (594M vs 15M for $bs=128$) but don’t proportionally increase latency improvement, confirming that page faults are off the critical path.
- **Model loading benefits modestly** (+4–6%) — disk I/O and deserialization dominate, not memory allocation.
- **THP always is safe and mildly beneficial for ML workloads** — unlike database and webserver workloads where THP compaction caused performance regressions, ML workloads use large contiguous allocations that naturally align with huge pages, avoiding compaction overhead.
- **Huge pages are not universally beneficial, even when metrics improve dramatically.** The ML workload is the clearest example: massive improvements in memory metrics don’t translate to proportional application-level gains when the workload is compute-bound.

VIII. CROSS-WORKLOAD COMPARISON

Table XXI summarizes the page size impact across all three workloads, demonstrating that the benefit of huge pages is highly workload-dependent.

TABLE XXI: Cross-Workload Page Size Impact

Aspect	Synthetic (Random)	Web Server	Database	ML Workload
Page size winner	2MB (+56%)	4KB (+33–133%)	4KB (+9–42%)	2MB (+1–5%)
Access pattern	Truly random, user-space	Kernel (sendfile)	Index traversal	Large contiguous tensors, BLAS
Page faults (4KB)	High (262K)	Very low (~ 65)	Low ($\sim 1K$ – $3K$)	Extremely high ($\sim 594M$ peak)
TLB miss reduction	99.7% with 2MB	Mixed / worse	No clear benefit	77–98% reduction with 2MB
Why 4KB wins	N/A	THP overhead, dTLB	THP overhead, buffer pool	N/A (compute-bound limits gains)

Key insights from the cross-workload comparison:

- 1) **Only purely random, user-space memory access benefits from huge pages.** The synthetic random workload—where each access touches a different page—is the only scenario with clear speedup.
- 2) **Kernel-mediated access (sendfile) bypasses TLB benefits.** nginx’s use of `sendfile()` means data never passes through user-space TLB entries, negating the advantage of huge pages.
- 3) **Application-level buffering masks page faults.** PostgreSQL’s buffer pool absorbs most memory accesses before they reach the OS, making page size nearly irrelevant for page fault behavior.
- 4) **THP always mode is harmful for server workloads.** The background compaction by `khugepaged` consistently degrades performance for both nginx and PostgreSQL.
- 5) **madvise mode is the correct default.** It allows applications to opt-in to huge pages only where beneficial.
- 6) **ML workloads see massive memory metric improvements but limited speedup.** Despite 95–

99% reductions in page faults and 77–98% reductions in dTLB misses, end-to-end performance improves only 1–5%.

- 7) **Compute-bound workloads diminish the impact of memory optimizations.** In ML inference, BLAS-dominated matrix operations dominate runtime, so reduced address translation overhead has minimal effect on wall time.
- 8) **Contiguous allocation patterns strongly favor huge pages.** Large tensor allocations (hundreds of MB) align naturally with 2 MB pages, eliminating most page faults and TLB pressure.
- 9) **Batch size scaling amplifies memory effects without proportional latency gains.** Larger batches dramatically increase page faults under 4 KB pages, but do not significantly change runtime due to compute dominance.
- 10) **Huge pages are most beneficial when memory overhead is on the critical path.** ML workloads demonstrate that even extreme improvements in memory metrics may not translate to performance gains unless memory is the bottleneck.

IX. CONCLUSIONS

This study provides a comprehensive, workload-aware characterization of how page size influences page fault behavior and application performance. Our key conclusions are:

- 1) **Huge pages (2MB) universally reduce page faults and TLB misses,** but the performance impact depends entirely on the memory access pattern. Page faults decreased by $\sim 407\times$ across all workloads with huge pages.
- 2) **Random user-space access benefits most:** the synthetic random workload achieved a $1.56\times$ speedup, with dTLB miss reduction of 99.7%. This result is directly applicable to workloads like hash tables and graph algorithms.
- 3) **Web server performance degrades with huge pages:** nginx throughput dropped 33–133% with 2MB pages due to THP compaction overhead, iTLB pollution, and internal fragmentation from small files.
- 4) **Database performance degrades with huge pages:** PostgreSQL OLTP throughput dropped 9–42% with 2MB pages because the internal buffer pool masks page fault benefits while THP overhead increases tail latency.
- 5) **ML workloads show massive memory improvements but modest performance gains:** page faults decrease by 95–99% and dTLB misses by 77–98%, yet inference latency improves only 1–5% because execution is dominated by compute-

intensive BLAS operations rather than memory access overhead.

- 6) **TLB miss reduction is the primary performance driver,** not page fault reduction. Page faults are a fixed one-time cost; TLB misses are ongoing during execution.
- 7) **THP madvise mode is the recommended configuration** for production systems, allowing workload-specific opt-in rather than system-wide huge page promotion.
- 8) **IPC is a useful diagnostic indicator:** low IPC (<0.5) signals TLB/memory bottlenecks where huge pages may help, while high IPC (>1.0) indicates spatial locality that already minimizes TLB overhead.
- 9) **Compute-bound workloads limit the benefits of huge pages:** the ML workload demonstrates that even extreme reductions in memory overhead do not translate into proportional application-level gains when computation dominates execution time.

These findings demonstrate that page size tuning must be informed by workload memory access patterns. Large pages are not universally beneficial, and indiscriminate use of THP `always` mode can significantly harm performance for server workloads.